

GUPAO TECH

AI全链路自主决策 workflow 引擎 实战课（上）

智枢·建筑施工方案AI生成与审核系统



讲师：朱博 | 时间：2026年4月21日

课程总览

CONTENTS

第1次课：RAG与 workflow 基础

- 模块1：行业痛点与技术必要性分析
- 模块2：项目全案架构设计思路
- 模块3：RAG全流程落地与 workflow 入门实践

第2次课：智能体与全项目整合

- 模块4：AI智能体开发原理与工具封装
- 模块5：复杂 workflow 核心要素与任务编排
- 模块6：全项目整合测试与生产级性能优化

GUPAO TECH

模块1：行业痛点与技术必要性分析

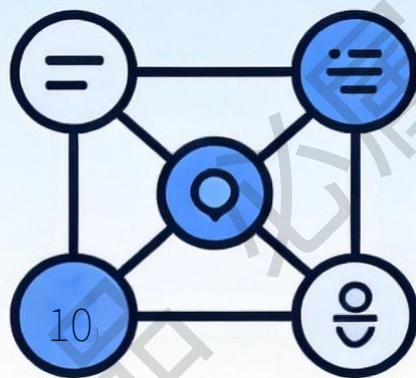
在咕泡·学懂 



你遇到这些问题了吗？



AI回答与文档内容不一致，
信息可靠性低



复杂任务无法自动化执行，
中途卡顿或报错



企业内部知识无法被
AI调用利用

这些问题如何解决？



大模型原生痛点



幻觉 (Hallucination)

生成看似逻辑严密、合理的内容，但实际上包含完全错误的信息或事实，即“一本正经地胡说八道”。



知识滞后 (Knowledge Cutoff)

训练数据存在固定的截止时间窗口，无法获取和学习训练完成后产生的最新信息、新闻或数据。



无自主决策能力

只能被动响应和回答用户提出的问题，缺乏主动规划、拆解任务和执行复杂逻辑的能力，无法独立完成 workflow。



流程不可控

生成过程属于“黑盒”机制，中间步骤不可见，无法对推理路径进行追溯、调试，也难以将特定的生成逻辑稳定复用。



建筑行业专属灾难：合规是生命线



规范条款错误

大模型生成的方案中包含不存在的规范编号或错误的技术参数，导致方案基础依据无效。



安全措施缺失

遗漏深基坑、脚手架等关键环节的安全防护规范，极易在实际施工中引发严重的安全事故。



工艺参数失真

生成的混凝土配合比、钢筋搭接长度等施工工艺参数严重偏离实际工程要求，无法落地执行。



审核标准不一

不同专业审核人员对同一方案的合规性理解存在差异，导致方案需要反复修改，严重拖慢工期。



三大核心技术定义与分工



RAG

检索增强生成

💡 核心解决问题

有效解决大模型“幻觉”问题，实现私有知识库（建筑国标）的精准调用。

🔗 本项目中的作用

自动检索建筑国标与行业规范，为方案生成提供权威依据，确保内容的合规性与准确性。



智能体 Agent

自主决策 · 工具调度

🧠 核心解决问题

解决任务拆解与工具使用逻辑，根据用户输入意图，自主判断并调度最优工具。

👤 本项目中的作用

作为系统的“大脑”，自动理解用户的设计需求，决定是否需要调用RAG检索规范，并将结果智能整合。



AI 工作流

流程标准化 · 可追溯

📋 核心解决问题

解决复杂业务的执行混乱问题，将碎片化的步骤串联成标准化、可监控、可回溯的流水线。

🏢 本项目中的作用

固化“编写-审核-修改-归档”的全流程，确保方案产出的每一个环节都有记录，实现全链路可追溯。



三者融合的核心价值

可靠知识 (RAG) + 自主决策 (智能体) + 标准流程 (工作流) = 生产级AI系统



效率提升

方案编制时间从3-5天大幅缩短至10分钟以内，实现业务响应的极速化。



合规保障

确保方案内容严格符合国标与行标规范，合规率显著提升至95%以上。



流程可控

方案的编审过程全程留痕，关键节点数据可查，便于企业统一管理和追溯。



知识沉淀

将企业内部的规范文档和过往优秀方案结构化沉淀，实现资产的持续复用。



一体化项目通用应用场景



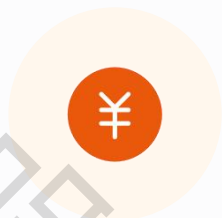
建筑行业 · 全周期赋能

覆盖施工方案制定、安全与技术交底、竣工验收报告生成等核心环节，标准化输出技术文档。



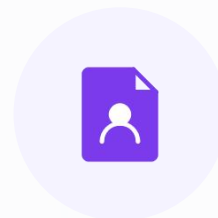
法律行业 · 智能合规

支持海量合同审查、深度法律案例分析、法律文书自动起草，大幅降低人工检索与撰写成本。



金融行业 · 风控研报

自动化生成行业研究报告，实时进行业务合规审查与风险动态评估，辅助投资决策分析。



通用办公 · 提效减负

实现周报/月报自动生成、会议纪要智能整理及企业知识库问答，释放员工基础事务性工作压力。

你的工作中，哪些重复的文本工作可以用这套方案解决？



文档与法规查阅

是否存在大量需要查阅
内部文档或外部法规的场景？



流程化审批审核

是否有一些固定的、
流程化的审批或审核工作？



专家经验沉淀

是否希望将专家的隐性经验
沉淀为团队可复用的知识？

GUPAO TECH

模块2：项目全案架构设计 思路

在咕泡·学懂 





项目基本信息与核心需求



项目名称

建筑施工方案文本AI生成与审核系统 —— 专注于施工领域的智能化文档处理平台



核心约束

纯文本聚焦

剔除多模态、图文生成等高复杂度需求，专注文本处理

本地部署架构

所有数据存储与模型计算均在本地服务器闭环完成

数据自主可控

确保企业核心数据安全，全生命周期数据“不落云”



核心功能

规范RAG检索

精准匹配施工规范知识库

AI方案生成

一键生成标准化方案初稿

AI合规审核

智能校验条款合规与风险

标准化 workflow

编写-审核-归档全流程管控



系统总体架构图（五层架构）



用户访问层

Web浏览器端：提供统一的用户交互入口



应用服务层

核心业务逻辑封装：用户权限管理、方案全生命周期管理、AI生成/审核服务、业务工作流程编排



AI 引擎层 (核心)

智能算力中心：大模型API调用、文本RAG检索增强引擎、智能体(Agent)调度、自动化 workflow 引擎



数据存储层

多模态数据持久化：MySQL存储结构化业务数据、Chroma向量库存储规范文档向量、本地文件系统存储源文件



基础设施层

本地PC服务器集群：提供基础计算与运行环境



本次课程 · 核心实现范围

🎯 重点突破：AI 引擎层核心组件

- **RAG 检索增强引擎**：实现文档加载、分割、向量化存储与相似度检索的完整流程。
- **基础 workflow 引擎**：搭建简单的任务调度与执行链路，串联AI生成与审核节点。

🔗 基础支撑：应用服务层

实现**方案管理模块**的基础功能（如方案的创建、列表展示），以及简单的用户权限拦截逻辑，为AI能力提供业务载体。

💡 **目标**：构建一个具备基础智能推理能力的**最小可行性系统(MVP)**



全流程链路图（核心）

用户输入工程信息

智能体意图识别

调度RAG检索规范

生成施工方案

AI审核 / 分支归档



意图识别模块

基于大模型与定制Prompt工程，精准理解用户输入的工程需求与核心意图。



RAG 规范检索

集成Chroma向量数据库，通过语义相似度搜索，快速匹配并召回相关工程规范条款。



施工方案生成

大模型结合用户意图与RAG检索到的规范结果，生成符合标准、逻辑严谨的施工方案。



智能合规性审核

利用大模型+Prompt对生成的方案进行自动审查，重点检查合规性、逻辑冲突及参数准确性，确保方案质量。



智能 workflow 分支

基于自定义业务逻辑判断审核结果：方案通过则自动归档；未通过则返回修改环节，形成完整的闭环工作流。



项目代码结构总览



.env 配置文件

集中管理敏感信息，如API密钥、数据库连接密码等，确保安全性。



app.py 项目主入口

项目的启动文件，负责整合RAG、Agent等所有功能模块，实现统一调度。



data/ 文档资源库

专门用于存放建筑行业规范、标准PDF文档，作为知识库的原始数据源。



rag/ 知识库检索

包含文档加载切片、向量库初始化入库及语义检索功能，是课程第一阶段的核心内容。



agent/ 智能体决策

基于大语言模型实现任务规划、工具调用与复杂推理，将在课程第二阶段完成开发。



workflow/ 业务编排

负责处理复杂任务的多步骤流程控制、状态管理与任务分发，分两次课程迭代完成。



核心设计思想（方法论）



分治思想

将复杂的编审任务，拆分为单一职责的节点（如检索、生成、审核），降低系统复杂度。



角色分离

决策（智能体）、检索（RAG）、流程（工作流）各司其职，实现关注点分离，降低耦合度。



状态可控

全流程通过统一字典传递状态数据，确保任务的每一步执行结果都可追溯、可调试、可干预。



可复用性

模块之间保持低耦合设计，核心逻辑与业务细节分离，可快速迁移至其他行业的文本生成/审核场景。



开发环境与技术栈



核心开发语言

Python 3.10+

作为主流的胶水语言，拥有丰富的第三方库生态，语法简洁，非常适合大模型应用的快速原型开发与迭代。



核心依赖组件

LangChain: 大模型应用开发框架，简化 RAG、智能体开发。

LangGraph: 基于LangChain的工作流引擎，用于构建多智能体协作和复杂流程。

ChromaDB: 轻量级、易于部署的本地向量数据库。

其他: 支持文档处理的python-docx/PyPDF2等。



大模型接入

兼容 OpenAI 格式 API

不绑定具体模型厂商，可灵活替换为任意支持该协议的国产大模型（如智谱、DeepSeek等），适配性极强。



两次课开发任务拆分

第一次课：基础构建



环境搭建

配置Python环境，安装项目所需的依赖包。



RAG 全流程实现

完成文档加载、文本切片、向量化、向量入库及检索召回。



基础 workflow 对接

定义核心 workflow 节点，实现RAG模块与 workflow 引擎的基础串联。

第二次课：智能与整合



智能体 Agent 开发

实现用户意图识别、任务决策逻辑，以及多工具的自动调用。



复杂 workflow 编排

设计并实现包含条件分支、循环逻辑的复杂内容编审自动化流程。



全项目整合与优化

将所有独立模块串联为完整系统，进行性能调优与生产级部署准备。

GUPAO TECH

模块3：RAG全流程落地与工 作流入门实践

在咕泡·学懂 





RAG核心流程回顾

文档加载 → 文本切片 → 文本向量化 → 向量库存储 → 相似度检索 → 答案生成



01. 文档加载

读取PDF/Word格式的建筑规范，将非结构化文档转换为可处理的文本数据。



02. 文本切片

将冗长的文档切分成固定长度的文本小块，适配大模型的上下文窗口限制。



03. 文本向量化

利用Embedding模型将文本块转换为高维数值向量，精准捕捉文本的深层语义。



04. 向量存储

将生成的所有向量持久化存入专业向量数据库中，建立可快速检索的知识库。



05. 相似检索

将用户查询向量化后，在向量库中进行余弦相似度计算，召回最相关的文本块。



06. 答案生成

将检索到的相关文本作为上下文，结合用户问题，让大模型生成准确的回答。



开发环境搭建 (5分钟)

01. 安装依赖包

```
pip install python-dotenv  
langchain chromadb pymysql  
python-docx PyPDF2
```

打开终端，执行上方命令，安装项目运行所需的所有Python第三方库。

02. 创建 .env 配置

```
# 配置大模型与数据库  
LLM_API_KEY = "你的密钥"  
LLM_BASE_URL = "API地址"  
MYSQL_HOST = "localhost"  
MYSQL_USER = "root"  
MYSQL_PWD = "你的密码"
```

在项目根目录新建该文件，集中管理敏感配置信息，避免硬编码。

03. 初始化项目目录

- docs (文档)
- src (核心代码)
- data (数据集)
- main.py (入口)

根据架构设计，手动创建标准的项目文件夹结构和必要的空Python文件，构建项目骨架。



文档加载与文本切片

📁 文件路径: rag/load_standard.py

```
from langchain.document_loaders import PyPDFLoader, Docx2txtLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

def load_and_split_document(file_path):
    """加载PDF/Word文档并进行文本切片，核心参数: chunk_size=1000, overlap=100"""
    loader = PyPDFLoader(file_path) # 切换Docx2txtLoader可加载Word
    docs = loader.load() # 加载文档内容

    splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
    chunks = splitter.split_documents(docs) # 执行切片
    return chunks

if __name__ == "__main__":
    chunks = load_and_split_document("../data/建筑施工规范.pdf")
    print(f"✅ 成功切分 {len(chunks)} 个文本块，首块内容: {chunks[0].page_content[:100]}...")
```




向量化与向量库入库

核心实现逻辑 (vector_db.py)



嵌入模型初始化

使用 `HuggingFaceEmbeddings` 加载轻量级开源模型 `all-MiniLM-L6-v2`，完成文本向量化转换。



Chroma 向量库配置

初始化 Chroma 客户端，指定集合名与本地持久化目录 `./chroma\_db`，构建本地向量存储环境。



文档块批量入库

调用 `add\_documents` 将切分好的文本块存入库中，执行 `persist()` 将向量数据持久化到本地磁盘。



测试验证

加载PDF切片数据，运行脚本生成本地向量库。

rag/vector_db.py # 核心代码实现

```
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma; import os

def init_chroma_db(): # 初始化向量库
    embed = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    db = Chroma(collection_name="standard",
embedding_function=embed,
persist_directory="./chroma_db")
    return db

def add_docs(chunks): # 文档入库
    db = init_chroma_db(); db.add_documents(chunks)
    db.persist(); print("✅ 文档已成功入库！")

if __name__ == "__main__": # 测试入口
    from load_standard import load_and_split
    chunks = load_and_split("../data/规范.pdf");
    add_docs(chunks)
```



RAG检索功能实现

文件路径: rag/rag_retriever.py

```
# 导入向量库初始化函数
from vector_db import init_chroma_db

def rag_search(query, top_k=3):

    """ 根据查询词进行RAG检索，返回最相关的top_k个结果 """
    db = init_chroma_db()# 初始化向量数据库
    retriever = db.as_retriever(search_kwargs={"k": top_k})# 转为检索器
    docs = retriever.get_relevant_documents(query)# 执行检索
    content = "\n".join([doc.page_content for doc in docs])

    return content

if __name__ == "__main__":
    query = "钢筋连接工艺的验收标准是什么?"
    result = rag_search(query)
    print("检索到的规范内容: \n"+ result)
```



workflow 基础认知

什么是 workflow?

在本项目中，我们使用**LangGraph**来构建 workflow。LangGraph 是一个基于状态图 (StateGraph) 的库，用于构建多智能体和复杂 workflow。它允许我们将任务定义为节点 (Nodes)，并通过边 (Edges) 连接它们，形成一个有向图。



节点 (Node)

一个独立的功能单元，如“检索规范”、“生成方案”。在 LangGraph 中，节点是一个接收状态并返回状态的函数。



状态 (State)

在节点之间传递的数据，通常是一个字典。LangGraph 通过状态来管理整个流程的数据。



路由 (Router)

决定流程走向的逻辑。在 LangGraph 中，通过条件边 (Conditional Edges) 实现。



基础 workflow 节点定义

节点 1: RAG 检索 (node_rag_retrieve)

核心职责: 接收用户查询, 调用封装好的 `rag\_search` 工具进行知识库检索, 并将检索到的建筑规范标准更新到 workflow 状态字典中, 供下游节点使用。

```
def node_rag_retrieve(state):  
    query = state["query"]  
    retrieved_standard = rag_search(query)  
  
    return {"retrieved_standard": retrieved_standard}
```

💡 设计要点: 所有节点函数统一遵循 `输入字典 -> 处理 -> 更新字典 -> 返回字典` 的标准接口, 确保状态在 workflow 中无缝传递。

节点 2: 方案生成 (node_generate_plan)

核心职责: 初始化大模型实例, 利用上一节点检索到的规范和输入的工程信息构建 Prompt, 调用 LLM 生成详细的施工方案, 并更新状态。

```
def node_generate_plan(state):  
    llm = OpenAI(...)  
    prompt = f"根据规范{state['retrieved_standard']}和工程信息{state['project_info']}生成方案"  
    generated_plan = llm.predict(prompt)  
  
    return {"generated_plan": generated_plan}
```

⚙️ 关键细节: 通过环境变量 `dotenv` 加载 API Key 等敏感配置, 避免硬编码。Prompt 工程是决定生成方案质量的核心因素。



RAG+ workflow基础对接

文件: workflow/base_workflow.py (续) —— 核心逻辑编排

Workflow 核心函数

```
def run_base_workflow(project_info, query):  
    state = {"project_info": project_info, "query": query} # 初始化状态字典  
    state = node_rag_retrieve(state) # 节点1: 检索知识库  
    state = node_generate_plan(state) # 节点2: 生成施工方案  
    return state # 返回最终执行状态
```

本地测试调用

```
if __name__ == "__main__":  
    info, query = "某住宅楼钢筋工程方案", "详细说明连接工艺和验收标准"  
    result = run_base_workflow(info, query)  
    print("生成方案结果: ", result["generated_plan"])
```

RAG+ workflow基础对接 (使用LangGraph)

文件: workflow/base_workflow.py (续) —— 核心逻辑编排

```
from langgraph.graph import StateGraph, END
```

1. 定义状态结构

```
class State(TypedDict):  
    project_info: str  
    query: str  
    retrieved_standard: str  
    generated_plan: str
```

2. 创建图

```
workflow = StateGraph(State)
```

3. 添加节点

```
workflow.add_node("node_rag_retrieve", node_rag_retrieve)  
workflow.add_node("node_generate_plan", node_generate_plan)
```

4. 设置入口点

```
workflow.set_entry_point("node_rag_retrieve")
```

5. 添加边

```
workflow.add_edge("node_rag_retrieve",  
                  "node_generate_plan")  
workflow.add_edge("node_generate_plan", END)
```

6. 编译图

```
app = workflow.compile()
```

7. 运行

```
result = app.invoke({  
    "project_info": "某住宅楼钢筋工程",  
    "query": "详细说明钢筋连接工艺和验收标准"  
})  
print(result["generated_plan"])
```



效果测试与常见问题排查



核心测试案例

测试工程背景

某商业综合体幕墙工程施工方案（包含玻璃、石材、铝板幕墙）

核心检索指令

“详细说明玻璃幕墙的安装工艺流程和现场质量验收标准”

建议基于真实工程数据进行多轮全流程测试，以验证系统的稳定性与准确性。



问题一：检索结果与意图不相关



可能成因

文档切片尺寸过大/过小；嵌入模型与垂直场景的语义匹配度低。



优化方案

调整 chunk_size 参数进行调优；尝试使用领域适配的专业嵌入模型。



问题二：生成内容格式混乱/不规范



可能成因

Prompt 指令描述模糊，缺乏格式约束；基座大模型能力不足以支撑复杂逻辑。



优化方案

优化 Prompt 增加格式要求；升级至能力更强的大模型（如 GPT-4）。

第1次课小结

COURSE SUMMARY



已完成内容

- RAG全流程实现（加载、切片、向量化、入库、检索）
- 基础 workflow 节点定义与配置
- RAG检索结果与 workflow 的基础对接



核心成果

基于RAG检索增强技术，成功生成了带有具体规范条文依据的施工方案文本，实现了从“生成”到“生成+引用”的质的飞跃。



课后作业

- 数据扩展：新增2份不同专业（如结构/机电）的建筑工程规范到向量库
- Prompt优化：调整提示词结构，提升生成方案的逻辑合理性与可读性

THANKS

谢谢观看

在咕泡·学懂AI